

Date	15 JUNE 2026
Team ID	xxxxxx
Project Name	Sustainable Waste Management Assistant Using Generative AI
Maximum Marks	5 Marks

Code-Layout, Readability and Reusability:
Well-structured code is essential for maintainability and collaboration. Document the standards, patterns, and principles your team followed to ensure the codebase is clean, readable, and highly reusable. Provide specific examples of how these practices were implemented in your project.

Code Quality & Structure Details

	\
1 Folder Directory Structure	/ Organized code into logical layers: /frontend/components, /frontend (React components), /backend /backend/routes, (Flask APIs), /database (Firestore configs), /utils/helpers.py /utils (helpers)
2 Naming Conventions	Used camelCase for variables/functions, scanHistory.js, PascalCase for React components, and WasteClassifier.jsx, snake_case for Python files data_loader.py
3 Code Readability Comments	Added docstrings for Python functions, /backend/classifier.py includes & inline comments for complex logic, and docstrings explaining ML JSDoc-style comments for utility functions pipeline

4	Modularity & Reusability	Applied DRY principle by creating reusable React components (Button, Modal, Chart) and modular Python functions for classification and database operations	Button.jsx reused across 5 pages, classifier.py functions reused in multiple routes
5	Formatting & Linting Tools	Configured Prettier (JS/React) and ESLint for frontend, Black and Flake8 for Python backend	.eslintrc.json, .prettierrc, pyproject.toml
6	Error Handling Patterns	Centralized error handling in Flask using error middleware; standardized error messages for frontend API calls	server.py error middleware, api.js error handler for failed requests